

# Resoluções Didáticas e Científicas —

## Capítulo 4 — Parte 3

**Obra:** C: Como Programar, 6ª Edição

**Autores:** Paul Deitel & Harvey Deitel

**Estratégia de Controle:** Divisão Estratégica (Parte 3: Exercícios de 4.26 a 4.35) para assegurar o cumprimento integral de todas as diretrizes pedagógicas, fornecendo códigos funcionais exaustivos, sem omissões e focados estritamente na matéria cumulativa.

### Apresentação Pedagógica

Olá! Dando sequência à nossa cobertura de alta fidelidade para o Capítulo 4, apresento a **Parte 3** do nosso guia de resoluções. Continuamos a explorar a fundo o potencial das estruturas de controle de fluxo, o comportamento de testes iterativos lógicos e a modelagem algorítmica rigorosa baseada exclusivamente no conteúdo do livro até este ponto. Conforme suas instruções estabelecidas e mantidas até o final do nosso trabalho, todos os trechos de código LaTeX estrutural foram eliminados, priorizando uma prosa técnica limpa e a implementação de códigos C puros prontos para compilação.

---

## Resoluções Detalhadas e Códigos Práticos (4.26 a 4.35)

### Exercício 4.26 — Cálculo de Aproximação Estatística de Pi ( $\pi$ )

**Enunciado:** Escreva um programa que calcule o valor de  $\pi$  a partir da série infinita:  $\pi = 4 - 4/3 + 4/5 - 4/7 + 4/9 - 4/11 + \dots$  Exiba uma tabela que mostre o valor aproximado de  $\pi$  após cada termo da série. Quantos termos são necessários para obter 3,14? E 3,141? E 3,1415? e 3,14159?

**Análise de Engenharia:** Este problema utiliza uma série alternada (Série de Leibniz). Controlamos a oscilação de sinal (positivo/negativo) multiplicando o termo por uma variável sinalizadora que inverte a cada iteração, enquanto o denominador cresce de 2 em 2.

```
#include <stdio.h>
```

```
int main(void) {
    double pi = 0.0;
    double numerador = 4.0;
    double denominador = 1.0;
    int sinal = 1;
    int termo;
    int termos_314 = 0, termos_3141 = 0, termos_31415 = 0,
    termos_314159 = 0;
```

```

printf("Termo\tAproximação de Pi\n");
printf("-----\n");

/* Executa 150.000 iterações para observar a convergência da
precisão */
for (termo = 1; termo <= 150000; termo++) {
    pi += sinal * (numerador / denominador);
    denominador += 2.0;
    sinal = -sinal; /* Inverte o sinal para a próxima iteração */

    /* Exibe apenas os primeiros 10 termos para não sobrecarregar
a tela */
    if (termo <= 10) {
        printf("%d\t%.5f\n", termo, pi);
    }

    /* Rastreia o número de termos necessários para cada nível de
precisão */
    if (termos_314 == 0 && (int)(pi * 100) == 314) {
        termos_314 = termo;
    }
    if (termos_3141 == 0 && (int)(pi * 1000) == 3141) {
        termos_3141 = termo;
    }
    if (termos_31415 == 0 && (int)(pi * 10000) == 31415) {
        termos_31415 = termo;
    }
    if (termos_314159 == 0 && (int)(pi * 100000) == 314159) {
        termos_314159 = termo;
    }
}

printf("\n--- Análise de Convergência ---\n");
printf("Termos para alcançar 3.14: %d\n", termos_314);
printf("Termos para alcançar 3.141: %d\n", termos_3141);
printf("Termos para alcançar 3.1415: %d\n", termos_31415);
printf("Termos para alcançar 3.14159: %d\n", termos_314159);

return 0;
}

```

## Exercício 4.27 — Trios Pitagóricos (Busca por Força Bruta)

**Enunciado:** Escreva um programa para encontrar todos os trios pitagóricos para `lado1`, `lado2` e a `hipotenusa`, todos não maiores que 500. Use um laço for triplamente aninhado para testar todas as possibilidades.

```
#include <stdio.h>
```

```

int main(void) {
    int lado1, lado2, hipotenusa;
    long conta_trios = 0;

    printf("Lado 1\tLado 2\tHipotenusa\n");
    printf("-----\n");

    /* Força bruta triplamente aninhada limitando o espaço de busca em
500 */
    for (lado1 = 1; lado1 <= 500; lado1++) {
        for (lado2 = lado1; lado2 <= 500; lado2++) { /* lado2 inicia
em lado1 para evitar duplicatas espelhadas */
            for (hipotenusa = 1; hipotenusa <= 500; hipotenusa++) {

                /* Verifica a equação fundamental de Pitágoras: a^2 +
b^2 = c^2 */
                if ((lado1 * lado1) + (lado2 * lado2) == (hipotenusa *
hipotenusa)) {
                    printf("%d\t%d\t%d\n", lado1, lado2, hipotenusa);
                    conta_trios++;
                }
            }
        }
    }

    printf("\nTotal de trios pitagóricos únicos encontrados: %ld\n",
conta_trios);
    return 0;
}

```

**Dica de Otimização 4.3:** Definir o escopo inicial do laço do lado2 igual ao valor corrente de lado1 elimina triângulos geometricamente idênticos que apenas inverteriam a ordem dos catetos (ex: 3, 4, 5 e 4, 3, 5), reduzindo o tempo de processamento pela metade.

## Exercício 4.28 — Sistema de Cálculo de Folha de Pagamento Semanal

**Enunciado:** Escreva um programa que calcule o pagamento semanal de cada funcionário de uma empresa com base em seus códigos de cargo: Gerentes (1 - salário fixo), Trabalhadores por hora (2 - salário fixo até 40 horas e hora extra de 1.5), Trabalhadores por comissão (3 - R\$ 250 fixos + 5.7% das vendas) e Trabalhadores por produção (4 - valor fixo por item produzido).  
#include <stdio.h>

```

int main(void) {
    int codigo;
    double salario, valor_hora, horas, vendas, valor_item, itens;

```

```

    printf("Digite o código do cargo (1:Gerente, 2:Por Hora,
3:Comissão, 4:Produção, -1 para encerrar): ");
    scanf("%d", &codigo);

    while (codigo != -1) {
        switch (codigo) {
            case 1:
                printf("Digite o salário semanal fixo do gerente: ");
                scanf("%lf", &salario);
                printf("Salário semanal do Gerente: R$ %.2f\n",
salario);
                break;

            case 2:
                printf("Digite o valor da hora e a quantidade de horas
trabalhadas: ");
                scanf("%lf%lf", &valor_hora, &horas);
                if (horas <= 40.0) {
                    salario = horas * valor_hora;
                } else {
                    salario = (40.0 * valor_hora) + ((horas - 40.0) *
valor_hora * 1.5);
                }
                printf("Salário semanal do Trabalhador por Hora: R$
%.2f\n", salario);
                break;

            case 3:
                printf("Digite o montante bruto de vendas semanais:
");
                scanf("%lf", &vendas);
                salario = 250.0 + (vendas * 0.057);
                printf("Salário semanal do Trabalhador por Comissão:
R$ %.2f\n", salario);
                break;

            case 4:
                printf("Digite o valor por item e a quantidade de
itens produzidos: ");
                scanf("%lf%lf", &valor_item, &itens);
                salario = valor_item * itens;
                printf("Salário semanal do Trabalhador por Produção:
R$ %.2f\n", salario);
                break;

            default:
                printf("Código de cargo inválido!\n");
                break;
        }
    }

```

```

    }

    printf("\nDigite o próximo código de cargo (-1 para encerrar):
");
    scanf("%d", &codigo);
}

return 0;
}

```

## Exercício 4.29 — Leis de De Morgan (Equivalência Lógica)

**Enunciado:** As Leis de De Morgan permitem simplificar expressões lógicas negadas. Reescreva as quatro expressões dadas utilizando essas leis e mostre que mantêm o mesmo valor lógico.

**Substituições Estruturadas de Engenharia:**

- **Expressão Original A:** `!(x < 5 && y >= 7)`  
**Equivalência De Morgan:** `!(x < 5) || !(y >= 7) → (x >= 5 || y < 7)`
- **Expressão Original B:** `!(a == b || c != d)`  
**Equivalência De Morgan:** `!(a == b) && !(c != d) → (a != b && c == d)`
- **Expressão Original C:** `!((x <= 8) && (y > 4))`  
**Equivalência De Morgan:** `!(x <= 8) || !(y > 4) → (x > 8 || y <= 4)`
- **Expressão Original D:** `!((i > 4) || (j <= 6))`  
**Equivalência De Morgan:** `!(i > 4) && !(j <= 6) → (i <= 4 && j > 6)`

## Exercício 4.30 — Substituindo a Instrução Switch por Estruturas Condicionais

**Enunciado:** Reescreva o programa da Figura 4.7 substituindo a estrutura multi-seleção switch por uma árvore aninhada de instruções condicionais if...else.

```

#include <stdio.h>

int main(void) {
    int nota;
    int conta_a = 0, conta_b = 0, conta_c = 0, conta_d = 0, conta_f =
0;

    printf("Digite as notas letradas (A, B, C, D, F) [EOF para
encerrar]:\n");

    while ((nota = getchar()) != EOF) {
        /* Substituição completa do switch por blocos lógicos
equivalentes if/else */
        if (nota == 'A' || nota == 'a') {
            conta_a++;
        }
    }
}

```

```

        else if (nota == 'B' || nota == 'b') {
            conta_b++;
        }
        else if (nota == 'C' || nota == 'c') {
            conta_c++;
        }
        else if (nota == 'D' || nota == 'd') {
            conta_d++;
        }
        else if (nota == 'F' || nota == 'f') {
            conta_f++;
        }
        else if (nota == '\n' || nota == '\t' || nota == ' ') {
            /* Ignora silenciosamente os caracteres de formatação e
espaçamento */
        }
        else {
            printf("Nota incorreta informada. Digite uma nova
nota.\n");
        }
    }

    printf("\nTotais por nota:\n");
    printf("A: %d\nB: %d\nC: %d\nD: %d\nF: %d\n", conta_a, conta_b,
conta_c, conta_d, conta_f);
    return 0;
}

```

## Exercício 4.31 — Desenho de Losango com Loops Aninhados

**Enunciado:** Escreva um programa que imprima um losango de asteriscos. Você pode usar instruções de saída que imprimam um único asterisco ou um único espaço.

```
#include <stdio.h>
```

```

int main(void) {
    int linha, espaco, asterisco;
    int n = 5; /* Define o raio superior do losango */

    /* Parte Superior do Losango (Linhas de 1 a 5) */
    for (linha = 1; linha <= n; linha++) {
        for (espaco = 1; espaco <= n - linha; espaco++) {
            printf(" ");
        }
        for (asterisco = 1; asterisco <= (2 * linha) - 1; asterisco++)
        {
            printf("*");
        }
    }
}

```

```

        printf("\n");
    }

    /* Parte Inferior do Losango (Linhas de 4 decrescendo até 1) */
    for (linha = n - 1; linha >= 1; linha--) {
        for (espaco = 1; espaco <= n - linha; espaco++) {
            printf(" ");
        }
        for (asterisco = 1; asterisco <= (2 * linha) - 1; asterisco++)
        {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}

```

### Exercício 4.32 — Desenho de Losango Modificado com Entrada de Tamanho

**Enunciado:** Modifique o programa do Exercício 4.31 para ler um número ímpar no intervalo de 1 a 19 fornecido pelo usuário, definindo a quantidade total de linhas do losango.

```
#include <stdio.h>
```

```

int main(void) {
    int linha, espaco, asterisco, total_linhas;
    int n;

    printf("Digite um número ímpar entre 1 e 19 para o tamanho do
losango: ");
    scanf("%d", &total_linhas);

    /* Validação para garantir que o número é ímpar e está no
intervalo seguro */
    while (total_linhas < 1 || total_linhas > 19 || total_linhas % 2
== 0) {
        printf("Entrada inválida! Digite um valor ÍMPAR entre 1 e 19:
");
        scanf("%d", &total_linhas);
    }

    n = (total_linhas / 2) + 1; /* Ponto central do losango */

    /* Metade superior */
    for (linha = 1; linha <= n; linha++) {

```

```

        for (espaco = 1; espaco <= n - linha; espaco++) {
            printf(" ");
        }
        for (asterisco = 1; asterisco <= (2 * linha) - 1; asterisco++)
    {
        printf("*");
    }
    printf("\n");
}

/* Metade inferior */
for (linha = n - 1; linha >= 1; linha--) {
    for (espaco = 1; espaco <= n - linha; espaco++) {
        printf(" ");
    }
    for (asterisco = 1; asterisco <= (2 * linha) - 1; asterisco++)
{
    printf("*");
}
    printf("\n");
}

return 0;
}

```

## Exercício 4.33 — Conversor Decimal para Numeração Romana

**Enunciado:** Escreva um programa que imprima uma tabela de todos os números romanos equivalentes aos decimais no intervalo de 1 a 100.

```
#include <stdio.h>
```

```

int main(void) {
    int i;
    printf("Dec\tAlgarismo Romano\n");
    printf("-----\n");

    for (i = 1; i <= 100; i++) {
        printf("%d\t", i);
        int temp = i;

        /* Processa a casa das Centenas (Apenas o valor 100 neste
escopo) */
        if (temp == 100) {
            printf("C");
            temp = 0;
        }
    }
}

```



```

/* Processa a casa das Dezenas (90, 50, 40, 10) */
if (temp >= 90) { printf("XC"); temp -= 90; }
else if (temp >= 50) { printf("L"); temp -= 50; }
else if (temp >= 40) { printf("XL"); temp -= 40; }

while (temp >= 10) { printf("X"); temp -= 10; }

/* Processa a casa das Unidades (9, 5, 4, 1) */
if (temp == 9) { printf("IX"); temp -= 9; }
else if (temp >= 5) { printf("V"); temp -= 5; }
else if (temp == 4) { printf("IV"); temp -= 4; }

while (temp >= 1) { printf("I"); temp -= 1; }

printf("\n");
}

return 0;
}

```

### Exercício 4.34 — Substituição de Tabelas Gráficas (Conversor Hexadecimal)

**Enunciado:** Escreva um programa que mostre tabelas contendo os equivalentes hexadecimais de números decimais. (Abordagem manual ou via formatação).

```
#include <stdio.h>
```

```

int main(void) {
    int num;
    printf("Decimal\tHexadecimal\n");
    printf("-----\n");

    for (num = 1; num <= 20; num++) {
        /* %X exibe automaticamente o equivalente hexadecimal em caixa
alta */
        printf("%d\t%X\n", num, num);
    }

    return 0;
}

```

### Exercício 4.35 — Remoção de Instruções de Interrupção Break

**Enunciado:** Elimine a instrução break do programa da Figura 4.11 sem alterar seu funcionamento lógico ou de exibição.

**Estratégia de Engenharia:** Substituímos a interrupção abrupta modificando a própria condição do laço for por meio de uma variável sinalizadora ou integrando um teste condicional interno que bloqueia novas execuções indesejadas.

```
#include <stdio.h>
```

```
int main(void) {
    int x;
    int continuar = 1; /* Variável de controle lógico estruturado */

    /* O laço continua ativo apenas enquanto x <= 10 E a flag
    continuar for verdadeira */
    for (x = 1; x <= 10 && continuar == 1; x++) {
        if (x == 5) {
            continuar = 0; /* Desativa o laço estruturalmente sem usar
            break */
        } else {
            printf("%d ", x);
        }
    }

    printf("\nLaço encerrado de forma limpa no valor x = %d\n", x);
    return 0;
}
```

---

*As resoluções finais do Capítulo 4 prosseguirão e serão concluídas na Parte 4.*